Lectura con filtro de simetría en dispositivos cuánticos de IBM: el filtro qg en veinte modelos de ruido de dispositivos mediante IBM Runtime

Vicente Humberto Monteverde
Universidad del Museo Social Argentino (UMSA), Argentina
ORCID 0000-0001-8884-4811

Borrador del 13 de octubre de 2026

Resumen

Los circuitos construidos con compuertas que conservan el peso de Hamming mantienen su estado en un solo sector de peso, de modo que los disparos que salieron del sector pueden descartarse en la lectura: es el filtro qg de la biblioteca de código abierto qang [1], una forma de verificación de simetría [2, 3]. Lo hacemos utilizable en dispositivos de IBM mediante IBM Runtime y Qiskit, y lo medimos en los modelos de ruido de veinte dispositivos de IBM. La biblioteca agrega un pase del transpilador de Qiskit que decide si el filtro se aplica, la corrección de lectura con los errores por qubit publicados de cada dispositivo, lecturas qg de los resultados del Sampler y del Estimator de IBM Runtime, extrapolación a ruido cero por plegado de compuertas que se combina con el filtro, una predicción de la fracción de disparos que conserva el filtro a partir de la calibración, la selección de qubits con esa predicción, un planificador de disparos y la red neuronal cuántica (QNN) de qang como un único trabajo parametrizado y como `SamplerQNN` de qiskit-machine-learning. Cada predicción se registró antes de su corrida y se informa como cumplida o fallida. En diez circuitos de cinco qubits y peso 1, el filtro baja el error del $qg_Z = \langle Z \rangle$ medido de 0,065–0,104 a 0,018–0,023 en tres dispositivos (0,013–0,022 con corrección de lectura), y el filtro con extrapolación a ruido cero llega a 0,011–0,015. En diecisiete dispositivos más, nunca usados antes, el filtro reduce el error 2,4–8,9 veces en todos (mediana de 3,1 veces con corrección de lectura). Elegir los qubits por la fracción conservada predicha conserva 6–17 puntos más de disparos que la ubicación por defecto de Qiskit en circuitos tipo cadena, evita un qubit publicado como que siempre lee 1, y el planificador cumplió una precisión objetivo de 0,02 en tres dispositivos. Para clasificación la ganancia es chica: la QNN recupera 5–6 de las 10 decisiones que le cuesta el ruido. Se encontraron dos errores de interoperabilidad, uno en qiskit-machine-learning y otro en la biblioteca. Todos los resultados son sobre modelos de ruido de dispositivos; no se informa ninguna corrida en hardware.

Palabras clave: mitigación de errores cuánticos; verificación de simetría; post-selección; IBM Quantum; aprendizaje automático cuántico.

1 Introducción

Cuando cada compuerta de un circuito conserva el número de excitaciones (el peso de Hamming del estado computacional), el ruido que cambia ese número saca disparos del sector en que empezó el circuito. Descartarlos es verificación de simetría por post-selección [2, 3]. En qang es el filtro qg, leído en la unidad $qg_Z = \langle Z \rangle \in [-1, 1]$ [1]; una nota complementaria lo estudia para redes neuronales cuánticas bajo relajación [4].

Este trabajo pregunta cuánto vale el filtro en dispositivos de IBM y da las herramientas para usarlo allí: todo corre mediante IBM Runtime [5] y Qiskit [6], aquí en el modo de prueba local sobre los modelos de ruido de los dispositivos (*fake backends*), y sin cambios en un dispositivo.

Cada experimento fijó sus circuitos, parámetros y predicciones en un commit antes de correr; los fracasos se informan como tales. Las secciones §137–§144 remiten a las notas de investigación del repositorio, donde cada número de este trabajo lo reproduce un script de `examples/`.

2 Métodos

Aplicabilidad. `WeightPreservationCheck` es un pase de análisis de Qiskit. Marca un circuito como conservador del peso cuando cada compuerta conserva el número de excitaciones (por nombre en compuertas como RZ, CZ, SWAP o XX+YY, y si no a partir de su matriz), y devuelve el peso inicial preparado por las compuertas X del comienzo. Se aplica al circuito lógico: después de transpilar, las compuertas nativas no conservan el peso una por una.

Lectura. Para un circuito transpilado a un dispositivo, `ibm_qg` identifica los qubits físicos medidos, toma sus errores de lectura asimétricos publicados $P(1|0)$, $P(0|1)$ e invierte el producto tensorial de las matrices de confusión por qubit sobre la distribución medida [7]. Devuelve qg_Z por qubit crudo, corregido por lectura, filtrado (los disparos de peso w de la distribución cruda) y filtrado después de la corrección (la parte de peso w de la cuasi-distribución corregida, renormalizada). Los qubits cuya matriz de confusión no se puede invertir se dejan sin corregir y se informan.

Fracción conservada y selección de qubits. `predict_kept_fraction` multiplica la probabilidad de que cada canal calibrado deje el peso intacto: una compuerta de un qubit con error e lo cambia con probabilidad e; una de dos qubits, con $5e/6$; la relajación durante el tiempo ocupado t de un qubit, con $(1 - e^{-t/T_1})\, w/n$; el desfase puro durante las compuertas, con $2z/3$, $z = (1 - e^{-t/T_\phi})/2$; y la lectura, con $\frac{w}{n}P(0|1) + (1 - \frac{w}{n})P(1|0)$. Sumar los disparos que vuelven al sector como para un estado totalmente mezclado da `kept_mixed_floor`. `best_chain` y `best_layout` eligen la ubicación inicial con la mayor predicción; `best_layout` evalúa el circuito tal como va a correr, ruteo incluido, y cuenta entre los candidatos la ubicación por defecto de Qiskit.

Extrapolación a ruido cero. La mitigación propia de IBM Runtime no tiene efecto en el modo local, así que `zne_qg` pliega cada compuerta de dos qubits ($G \to GG^\dagger G$, escala 3) [8], corre ambas escalas y extrapola linealmente a cero [9]. El plegado mantiene conservador del peso a un circuito que lo era, por lo que se combina con el filtro.

Planificación de disparos. Un qg_Z filtrado leído de Nk disparos conservados tiene error estándar $\sqrt{(1 - qg_Z^2)/(Nk)}$. `plan_shots` devuelve el N que lleva al peor qubit a un error objetivo con el k predicho, y el tiempo de dispositivo N(duración del circuito + retardo de repetición).

QNN. La QNN de peso 1 de qang [4] (un cargador de amplitudes y tres capas de compuertas RBS que conservan el peso, sobre cinco qubits acoplados en anillo) se escribe como un único circuito parametrizado de compuertas nativas XX+YY, $\mathrm{RBS}(t) = \mathrm{XXPlusYY}(2t, \pi/2)$, igual al simulador de la biblioteca hasta 10^{-9}. `run_qnn` envía todo un conjunto de prueba como un solo *pub* del Sampler. `qang_sampler_qnn` envuelve el mismo circuito como `SamplerQNN` de qiskit-machine-learning [10]. El modelo se entrena sin ruido en cuatro conjuntos de datos (iris, cancer, wine, digits; 189 entradas de prueba; 185 correctas sin ruido).

Circuitos de prueba. Diez circuitos aleatorios de cinco qubits y peso 1: X en el qubit 0 y dos capas de compuertas XX+YY entre vecinos con ángulos uniformes en [0,3; 2,8] (semillas

0–9). El error es el promedio de $|qg_Z - qg_Z^{\text{ideal}}|$ sobre qubits y circuitos. Se transpila con nivel de optimización 1 y semilla 137.

3 Resultados

3.1 Lectura en tres dispositivos (§137, §142)

	crudo	lectura	filtro	filtro + lectura	ZNE	filtro + ZNE
FakeBrisbane	0,104	0,074	0,023	0,022	0,053	0,012
FakeSherbrooke	0,065	0,038	0,019	0,016	0,034	0,011
FakeTorino	0,102	0,036	0,018	0,013	0,085	0,015

Tabla 1: Error medio de qg_Z, diez circuitos; 8000 disparos por circuito (y por escala de ruido en ZNE). Las cuatro primeras columnas son de §137 y las de ZNE de §142 (corridas distintas).

Las tres predicciones de §137 se cumplieron: la corrección de lectura baja el error crudo (entre 29 y 64%), el filtro rinde más que la corrección de lectura sola y ambos juntos son lo mejor (Tabla 1). La corrección de lectura sube la masa de peso 1 de 0,69–0,82 a 0,82–0,92: entre un tercio y dos tercios de los disparos que el filtro descartaría eran errores de lectura. Con ZNE (§142) el filtro rinde 2–7 veces más que ZNE solo (cumplida); ZNE bajó el error filtrado y corregido en dos dispositivos pero no en Torino, y la mejor combinación fue filtro + ZNE en dos dispositivos y filtro + lectura en Torino (así que esas dos predicciones fallaron).

3.2 Veinte dispositivos (§144)

En los otros dieciocho *fake backends* de al menos 100 qubits (dispositivos ECR, CZ y CX de 120 a 156 qubits), el filtro redujo el error 2,4–8,9 veces en cada uno de los diecisiete que corrieron (Fig. 1; mediana crudo / filtro + lectura de 3,12). Aun así, las tres predicciones fallaron tal como estaban escritas: un dispositivo se colgó (dos predicciones exigían los dieciocho), y filtro + lectura fue lo más bajo en 10 de 17, y el filtro solo en los otros 7, donde los errores de lectura son chicos y corregirlos agrega varianza. El cuelgue fue un error de la biblioteca: FakeCusco publica un qubit que siempre lee 1, el transpilador ubicó el circuito ahí y la inversa de lectura era singular. Después del arreglo, `best_layout` esquiva ese qubit y baja el error filtrado de 0,098 a 0,021 (exploratorio).

3.3 Predecir la fracción conservada y elegir qubits (§138, §141)

Sobre diez ubicaciones por dispositivo, la predicción ordenó bien las ubicaciones (Spearman 0,94–0,98, cumplida) y `best_chain` conservó 6–17 puntos más de disparos que la ubicación por defecto de Qiskit (cumplida), pero la predicción cruda quedó baja por 0,08–0,13 (fallida): ignoraba los disparos que vuelven al sector. Con el piso de estado mezclado, agregado después de puntuar, el error es 0,023–0,041. Para la QNN acoplada en anillo, `best_chain` evaluó un circuito distinto del que corrió y conservó menos disparos en dos dispositivos (§139); `best_layout` lo corrige: nunca fue peor que la ubicación por defecto y ganó 2,2 puntos donde eligió otra (§141, dos de tres predicciones cumplidas).

3.4 Planificación de disparos (§143)

Para un error objetivo de 0,02 el planificador pidió 3000–3400 disparos (0,8–0,9 segundos de dispositivo); en 30 repeticiones el error del peor qubit fue 0,021–0,023 (cumplida). La predicción por qubit quedó en promedio 1,4–1,6 veces baja (fallida): los qubits cerca de un polo ($|qg_Z| > 0{,}8$)

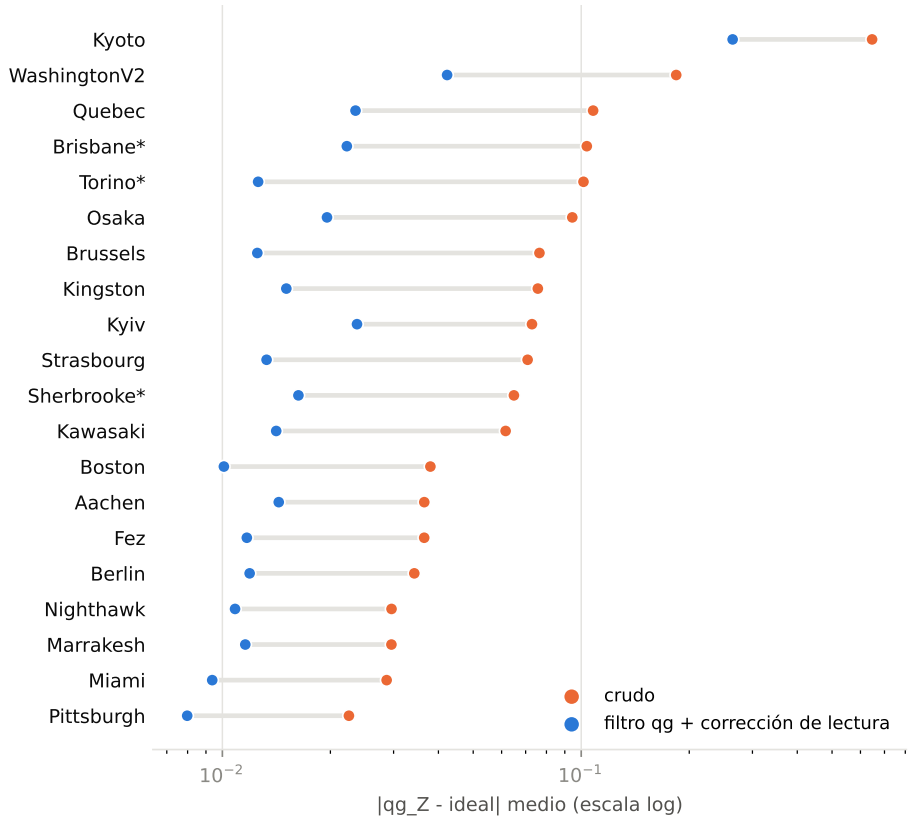


Figura 1: Error de qg_Z crudo y filtrado (+ corregido por lectura) en veinte modelos de ruido de dispositivos de IBM. Con asterisco: §137 (diez circuitos, 8000 disparos); el resto: §144 (cinco circuitos, 4000 disparos), dispositivos no usados antes.

quedan subestimados 2,4–3,1 veces, porque el ruido aleja su qg_Z de ± 1; el plan lo fija el peor qubit y aun así cumplió el objetivo.

3.5 La QNN (§139–§141)

	crudo	filtro	filtro + lectura
`run_qnn`, ubicación por defecto (§139)	545	550	549
`run_qnn`, `best_chain` (§139)	547	551	551
`run_qnn`, `best_layout` (§141)	545	549	550
`SamplerQNN` de qiskit-machine-learning (§140)	547	548	–

Tabla 2: Decisiones correctas de 567 (189 entradas en cada uno de tres dispositivos; 555 sin ruido), 1000 disparos.

La plantilla de un solo trabajo coincide con los circuitos por entrada (550 contra 551, cumplida) y qiskit-machine-learning reproduce el camino propio de la biblioteca (548, cumplida), pero el filtro recupera solo 5–6 de las 10 decisiones que cuesta el ruido, y la corrección de lectura no cambia ninguna decisión (Tabla 2). Corridas con la misma ubicación difieren hasta en dos decisiones, así que diferencias de ese tamaño son ruido de muestreo. Al envolver la QNN en qiskit-machine-learning 0.9.1 apareció que, con un *pass manager* de dispositivo, `SamplerQNN` lee los bits medidos en las posiciones de los qubits físicos de la ubicación, de modo que un circuito medido antes de transpilar lee cada disparo como 0 salvo que su ubicación sea $[0, \ldots, n-1]$; la

biblioteca transpila por su cuenta y descarta la ubicación, y el problema se informó al proyecto.

4 Discusión

Como herramienta de lectura, el filtro es robusto: en cada uno de los veinte modelos de ruido de dispositivos de IBM bajó el error de qg_Z en un factor de 2,4 o más, y se combina con la corrección de lectura y con la extrapolación a ruido cero. Cuál combinación es la mejor depende del dispositivo: la corrección de lectura ayuda donde los errores de lectura son grandes y agrega varianza donde son chicos. La calibración también dice, antes de gastar tiempo de dispositivo, cuántos disparos conservará el filtro, qué qubits conservan más y cuántos disparos necesita una precisión objetivo.

Los límites son igual de claros. La ganancia en exactitud de clasificación es chica, porque las decisiones del modelo están en su mayoría lejos del umbral. Nueve de las veintitrés predicciones fallaron tal como estaban escritas, varias por el análisis y no por el filtro: una predicción que ignoraba los disparos que vuelven, un modelo de error por qubit que ignoraba los polos y un experimento que exigía que corrieran todos los dispositivos. Y todos los resultados son sobre modelos de ruido de dispositivos, que tratan los errores de compuerta, relajación, desfase y lectura como canales independientes; no incluyen fugas, interferencia entre qubits ni deriva. El paso siguiente es una corrida en hardware.

Disponibilidad de código y datos

Todo está en la biblioteca de código abierto qang (licencia MIT), versión 0.9.0 o posterior: `https://github.com/Viny2030/qang`, `pip install qang`. El módulo es `qang.ibm` (con `qang.qiskit_ml`); cada sección tiene un script en `examples/` y su salida en `examples/data/`; `docs/ibm_quickstart.md` muestra el flujo de trabajo.

```
from qang import ibm
t = transpile(qc, backend) # qc: conserva el peso, medido
res = Sampler(mode=backend).run([t]).result()[0]
ibm.ibm_qg(res, t, backend, weight=1)["filtered_readout"]
```

Referencias

[1] Vicente Humberto Monteverde. The Qang (qg): A unified angular-probability unit and metric for parametric quantum circuit design. 2026. doi: 10.5281/zenodo.22832150.

[2] Xavier Bonet-Monroig, Ramiro Sagastizabal, M. Singh, and Thomas E. O'Brien. Low-cost error mitigation by symmetry verification. *Physical Review A*, 98:062339, 2018.

[3] Sam McArdle, Xiao Yuan, and Simon Benjamin. Error-mitigated digital quantum simulation. *Physical Review Letters*, 122:180501, 2019.

[4] Vicente Humberto Monteverde. Training quantum neural networks for T1-limited hardware: what the qg symmetry filter does and does not do, 2026. Companion note, `manuscript/qml/` in the repository.

[5] IBM Quantum. Qiskit IBM runtime client, version 0.50. `https://github.com/Qiskit/qiskit-ibm-runtime`, 2026.

[6] Ali Javadi-Abhari et al. Quantum computing with Qiskit, 2024.

[7] Sergey Bravyi, Sarah Sheldon, Abhinav Kandala, David C. Mckay, and Jay M. Gambetta. Mitigating measurement errors in multiqubit experiments. *Physical Review A*, 103:042605, 2021. doi: 10.1103/PhysRevA.103.042605.

[8] Tudor Giurgica-Tiron, Yousef Hindy, Ryan LaRose, Andrea Mari, and William J. Zeng. Digital zero noise extrapolation for quantum error mitigation. In *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 306–316, 2020. doi: 10.1109/QCE49297.2020.00045.

[9] Kristan Temme, Sergey Bravyi, and Jay M. Gambetta. Error mitigation for short-depth quantum circuits. *Physical Review Letters*, 119(18):180509, 2017. doi: 10.1103/PhysRevLett.119.180509.

[10] Qiskit community. Qiskit machine learning, version 0.9.1. https://github.com/qiskit-community/qiskit-machine-learning, 2026.